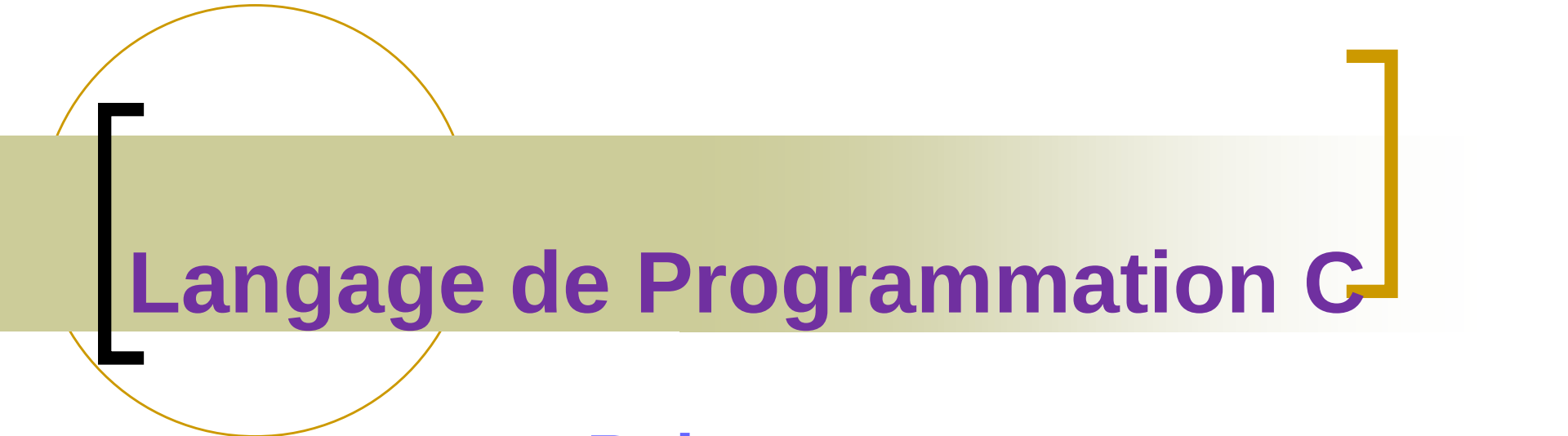


A decorative graphic consisting of a thin yellow circle on the left and a horizontal bar with a yellow-to-white gradient on the right. A large black '[' bracket is on the left, and a yellow ']' bracket is on the right, framing the title text.

Langage de Programmation C

Pointeurs

Département de la formation préparatoire ST
Ecole Nationale Polytechnique d'Oran

[**Partie 3**

- **Pointeurs**

Pointeurs : définition

■ Adressage de variables

Avant de parler de pointeurs, il est indiqué de brièvement passer en revue les deux modes d'adressage principaux:

1. Adressage direct
2. Adressage indirect

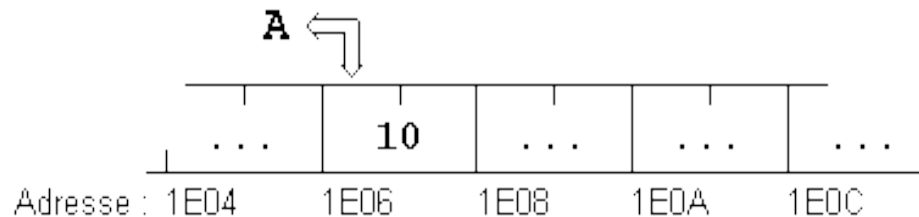
Pointeurs : définition

Adressage direct

- Dans la programmation, nous utilisons des variables pour stocker des informations.
- La valeur d'une variable se trouve à un endroit spécifique dans la mémoire interne de l'ordinateur.
- Le **nom** de la variable nous permet alors d'**accéder directement** à cette **valeur**.

Exemple:

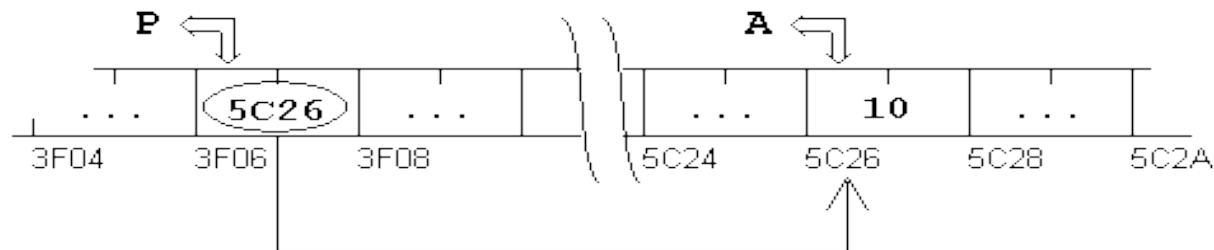
```
short A;  
A = 10;
```



Adressage direct: Accès au **contenu** d'une variable par le **nom** de la variable.

Pointeurs : définition

- **Adressage indirect**
- Si nous ne voulons ou ne pouvons pas utiliser le nom d'une variable **A**, nous pouvons copier l'adresse de cette variable dans une variable spéciale **P**, appelée **pointeur**. Ensuite, nous pouvons retrouver l'information de la variable **A** en passant par le pointeur **P**.
- **Exemple**
- Soit A une variable contenant la valeur 10 et P un pointeur qui contient l'adresse de A. En mémoire, A et P peuvent se présenter comme suit:



Adressage indirect: Accès au **contenu** d'une variable, en passant par un **pointeur qui contient l'adresse** de la variable.

Pointeurs : définition

■ On résume:

- Un pointeur est une variable spéciale qui peut contenir l'adresse d'une autre variable.
- Le nom d'une variable permet d'accéder *directement* à sa valeur (**adressage direct**).
- Un pointeur qui contient l'adresse de la variable, permet d'accéder *indirectement* à sa valeur (**adressage indirect**).

■ Remarque :

Les pointeurs et les noms de variables ont le même rôle: Ils donnent accès à un emplacement dans la mémoire interne de l'ordinateur. Il faut quand même bien faire la différence:

- Un **pointeur** est une variable qui peut 'pointer' sur différentes adresses.
- Le **nom d'une variable** reste toujours lié à la même adresse.

Intérêts des pointeurs

- Les pointeurs présentent de nombreux avantages :
 - Ils sont indispensables pour permettre le passage par référence pour les paramètres des fonctions
 - Ils permettent de créer des structures de données (listes et arbres) dont le nombre d'éléments peut évoluer dynamiquement , Ces structures sont très utilisées en programmation.
 - Ils permettent d'écrire des programmes plus compacts et efficaces

Opérateurs de manipulation des pointeurs

- Lors du travail avec des pointeurs, nous utilisons :
 - un opérateur '**adresse de**': **&** pour obtenir l'adresse d'une variable
 - un opérateur '**contenu de**': ***** pour accéder au contenu d'une adresse
 - d'une **syntaxe de déclaration** pour pouvoir déclarer un pointeur.
- L'opérateur '**adresse de**' : **&**
 - **&<NomVariable>** fournit l'adresse de la variable **<NomVariable>**
L'opérateur **&** nous est déjà familier par la fonction **scanf**, qui a besoin de l'adresse de ses arguments pour pouvoir leur attribuer de nouvelles valeurs.
- Exemple

```
int N;  
printf("Entrez un nombre entier : ");  
scanf("%d", &N);
```

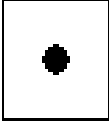
Attention !

- L'opérateur **&** peut seulement être appliqué à des objets qui se trouvent dans la mémoire interne, c.-à-d. à des variables et des tableaux. Il ne peut pas être appliqué à des constantes ou des expressions.

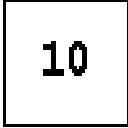
Opérateurs de manipulation des pointeurs

■ Représentation schématique

Soit P un pointeur non initialisé

`int *p;` **P :** 

et A une variable (du même type) contenant la valeur 10 :

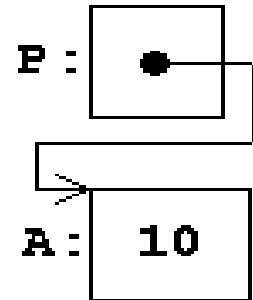
`int A=10;` **A :** 

Alors l'instruction

`P = &A;`

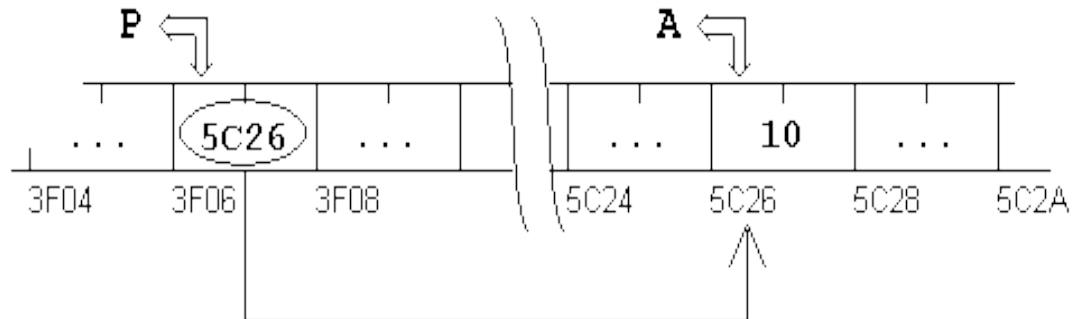
affecte l'adresse de la variable A à la variable P.

nous pouvons illustrer le fait que '**P pointe sur A**' par une flèche:



Opérateurs de manipulation des pointeurs

En mémoire, A et P se présentent :



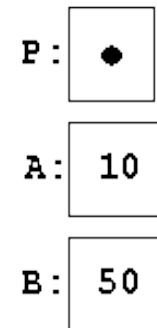
Opérateurs de manipulation des pointeurs

L'opérateur '**contenu de**' : *

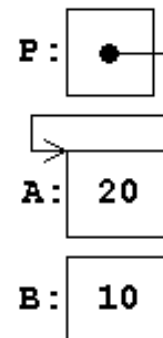
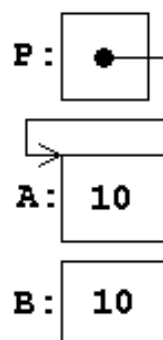
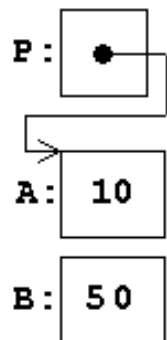
- ***<NomPointeur>** désigne le *contenu de l'adresse* référencée par le pointeur <NomPointeur>

- **Exemple**

Soit A une variable contenant la valeur 10,
B une variable contenant la valeur 50
et P un pointeur non initialisé:



- Après les instructions,
P = &A; - P pointe sur A,
B = *P; - le contenu de A (référéncé par *P) est affecté à B
***P = 20;** - le contenu de A (référéncé par *P) est mis à 20.



Déclaration d'un pointeur

- En C, chaque pointeur est limité à un type de donnée (même si la valeur d'un pointeur, qui est une adresse, est toujours un entier).
- Le type d'un pointeur dépend du type de la variable pointée. Ceci est important pour connaître la taille de la valeur pointée.
- On **déclare** un pointeur par l'instruction :
type *nom-du-pointeur ;
 - **type** est le type de la variable pointée
 - ***** est l'opérateur qui indiquera au compilateur que c'est un pointeur
- **Exemple :**
 - **int *pi; //pi est un pointeur vers une variable de type int**
 - **float *pf; //pf est un pointeur vers une variable de type float**
- **Rq:** la valeur d'un pointeur donne l'adresse du premier octet parmi les n octets où la variable est stockée

[Opérateurs de manipulation des pointeurs]

■ Exemple1 :

```
int *p;           //on déclare un pointeur vers une variable de type int
int i=15, j=30;   // deux variables de type int
p=&i;             // on met dans p, l'adresse de i (p pointe sur i)
printf("*p = %d \n",*p); //affiche : *p = 15
*p=25;           // met la valeur 25 dans la case mémoire pointée par p
                  // (i vaut 25 après cette instruction)

p=&j;             // p pointe sur j
i=*p;            // on affecte le contenu de p à i
                  // (i vaut 30 après cette instruction)
```

Opérateurs de manipulation des pointeurs

■ Exemple2 :

```
float a, *p;  
p=&a;  
printf("Entrez une valeur : \n");  
scanf("%f ",p); //supposons qu'on saisit la valeur 1.5  
printf("Adresse de a= %x, contenu de a= %f\n" , p,*p);  
*p+=0.5;  
printf ("a= %f\n" , a); //affiche a=2.0
```

■ **Remarque :** si un pointeur P pointe sur une variable X, alors *P peut être utilisé partout où on peut écrire X

- X+=2 équivaut à *P+=2
- ++X équivaut à ++ *P
- X++ équivaut à (*P)++ // les parenthèses ici sont obligatoires car l'associativité des opérateurs unaires * et ++ est de droite à gauche

Initialisation d'un pointeur

- A la déclaration d'un pointeur p, on ne sait pas sur quel zone mémoire il pointe. Ceci peut générer des problèmes :
 - `int *p;`
 - `*p = 10;` //provoque un problème mémoire car le pointeur p n'a pas été initialisé
- **Conseil :** Toute utilisation d'un pointeur doit être précédée par une initialisation.
- On peut initialiser un pointeur en lui affectant :
 - l'adresse d'une variable (Ex: `int a, *p1; p1=&a;`)
 - un autre pointeur déjà initialisé (Ex: `int *p2; p2=p1;`)
 - la valeur 0 désignée par le symbole NULL, défini dans `<stddef.h>`.
- **Ex:** `int *p; p=0;` ou `p=NULL;` (on dit que p pointe 'nulle part': aucune adresse mémoire ne lui est associé)
- **Rq:** un pointeur peut aussi être initialisé par une allocation dynamique 15 (voir fin du chapitre)

[on résume]

Après les instructions:

```
int A;  
int *P;  
P = &A;
```

- A** désigne le contenu de A
- &A** désigne l'adresse de A
- P** désigne l'adresse de A
- *P** désigne le contenu de A

En outre:

- &P** désigne l'adresse du pointeur P
- *A** est illégal (puisque A n'est pas un pointeur)

exercice

main()

```
{ int A = 1, B = 2, C = 3, *P1, *P2;
```

```
    P1=&A;
```

```
    P2=&C;
```

```
    *P1=(*P2)++;
```

```
    P1=P2;
```

```
    P2=&B;
```

```
    *P1 -=*P2;
```

```
    ++*P2;
```

```
    *P1*=*P2;
```

```
    A=++*P2**P1;
```

```
    P1=&A;
```

```
    *P2=*P1/=*P2;
```

```
}
```

- Donnez les valeurs de A, B,C,P1 et P2 après chaque instruction

	<u>A</u>	<u>B</u>	<u>C</u>	<u>P1</u>	<u>P2</u>
Init.	1	2	3	/	/
P1=&A	1	2	3	&A	/
P2=&C	1	2	3	&A	&C
*P1=(*P2)++	3	2	4	&A	&C
P1=P2	3	2	4	&C	&C
P2=&B	3	2	4	&C	&B
*P1-=*P2	3	2	2	&C	&B
++*P2	3	3	2	&C	&B
P1=*P2	3	3	6	&C	&B
A=++*P2**P1	24	4	6	&C	&B
P1=&A	24	4	6	&A	&B
*P2=*P1/=*P2	6	6	6	&A	&B

Opérations arithmétiques avec les pointeurs

- La valeur d'un pointeur étant un entier, certaines opérations arithmétiques sont possibles : ajouter ou soustraire un entier à un pointeur ou faire la différence de deux pointeurs
- Pour un entier *i* et des pointeurs *p*, *p1* et *p2* sur une variable de **type T**
 - ***p+i* (resp *p-i*) : désigne un pointeur sur une variable de type T. Sa valeur est égale à celle de *p* incrémentée (resp décrétementée) de *i*sizeof(T)*.**
 - ***p1-p2* : Le résultat est un entier dont la valeur est égale à (différence des adresses)/sizeof(T).**

Remarque:

- on peut également utiliser les opérateurs ++ et -- avec les pointeurs
- la somme de deux pointeurs n'est pas autorisée

Opérations arithmétiques avec les pointeurs

Exemple :

```
float *p1, *p2;  
float z =1.5;  
p1=&z;  
printf("Adresse p1 = %x \n",p1);  
p1++;  
p2=p1+1;  
printf("Adresse p1 = %x \t Adresse p2 = %x\n",p1,p2);  
printf("p2-p1 = %d \n",p2-p1);
```

- On suppose que `&p1=22ff44` (en hexadécimal)

- Affichage :

Adresse p1 = 22ff44

Adresse p1 = 22ff48

p2-p1=1

Adresse p2 = 22ff4c